

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 22

Внедрение глобального управления состоянием в React-приложение с
использованием Redux

Вариант 2 – лендинг IT-конференции

Выполнил: Клычева Г.

Группа: 310102

Репозиторий:

<https://github.com/karrinkl/devconf-react-redux.git>

Проверил: Прудник А.М.

1 ЦЕЛЬ РАБОТЫ И УСЛОВИЕ ЗАДАНИЯ

Цель работы состоит в получении практических навыков внедрения централизованного хранилища данных в React-приложение. Требуется вынести часть состояния компонентов, созданных в лабораторной работе № 19, на глобальный уровень и обеспечить предсказуемую синхронизацию данных между несвязанными компонентами.

По варианту 2 предметной областью выступает лендинг IT-конференции. Нужно создать срез состояния для списка избранных спикеров, добавить в карточку спикера кнопку, отправляющую действие добавления или удаления идентификатора, и реализовать компонент избранного, отображающий количество отмеченных докладов.

Проект продолжает работу № 19: компоненты, вёрстка и графические ассеты перенесены без изменений, добавлено хранилище и два подключённых к нему компонента. Условие допускает выбор между Redux Toolkit и Zustand, выбран первый вариант.

2 ВЫБОР СТЕЙТ-МЕНЕДЖЕРА

Оба допустимых инструмента решают одну задачу, но различаются объёмом архитектурных понятий и строгостью потока данных. Сопоставление приведено в таблице 1.

Таблица 1 – Сравнение Redux Toolkit и Zustand

Признак	Redux Toolkit	Zustand
Поток данных	Строгий: компонент отправляет действие, редюсер возвращает новое состояние	Свободный: функция-действие изменяет состояние напрямую
Подключение	Нужен Provider в корне дерева	Provider не требуется, хук импортируется напрямую
Инструменты отладки	Redux DevTools подключаются автоматически	Подключаются отдельным промежуточным обработчиком
Объём кода	Больше: срез, действия, селекторы, хранилище	Меньше: одно объявление хука
Порог входа	Выше, требуется понимание редюсеров и действий	Ниже, ближе к обычному useState

Выбран Redux Toolkit. Для учебной задачи важнее не краткость записи, а наглядность архитектуры: действие, редюсер и селектор существуют как отдельные сущности, и поток данных виден в коде целиком. Дополнительный довод - Redux DevTools, где каждое отправленное действие и состояние хранилища до и после него отображаются без дополнительной настройки. Условие задания также отмечает этот вариант как рекомендуемый.

3 АРХИТЕКТУРА И СТРУКТУРА ХРАНИЛИЩА

Поток данных в приложении односторонний и замкнут через хранилище. Компонент отправляет действие функцией `dispatch`, редюсер среза вычисляет новое состояние, подписанные через `useSelector` компоненты получают обновлённые значения и перерисовываются. Схема приведена на рисунке 1.

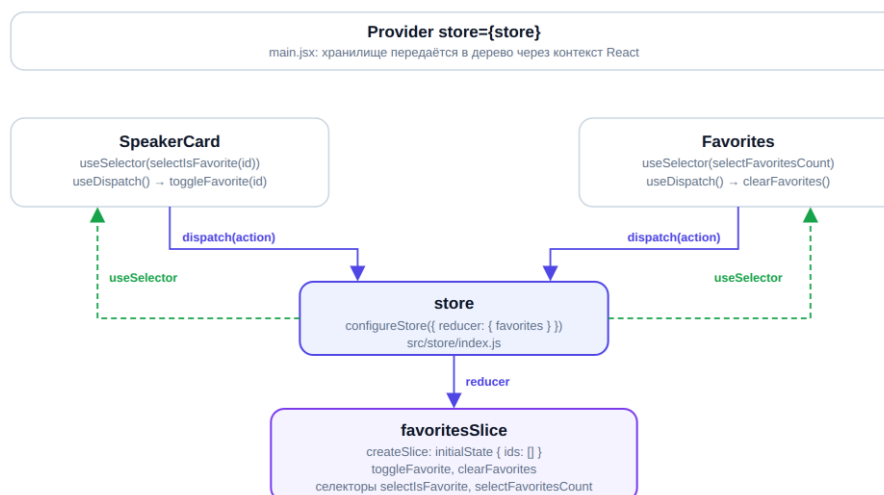


Рисунок 1 – Поток данных между компонентами и хранилищем

Сплошные стрелки обозначают отправку действий, пунктирные - чтение состояния селекторами. Прямых связей между SpeakerCard и Favorites на схеме нет: компоненты обмениваются данными только через хранилище.

Логика хранилища вынесена в отдельный каталог src/store и не содержит разметки. Файлы, добавленные и изменённые относительно ЛР № 19, перечислены в таблице 2.

Таблица 2 – Изменения в проекте относительно ЛР № 19

Файл	Статус	Содержание изменений
src/store/favoritesSlice.js	добавлен	Срез состояния: начальное состояние, редюсеры, селекторы
src/store/index.js	добавлен	Корневое хранилище, собранное функцией configureStore
src/components/Favorites/	добавлен	Компонент счётчика избранного и его файл стилей
src/main.jsx	изменён	Корневой компонент обернут в Provider
src/components/SpeakerCard/	изменён	Добавлена кнопка избранного, подключены useSelector и useDispatch
src/App.jsx	изменён	В заголовок секции спикеров добавлен компонент Favorites
package.json	изменён	Добавлены зависимости @reduxjs/toolkit и react-redux

Остальные файлы проекта перенесены из ЛР № 19 без правок, включая компоненты Accordion, InputField и RegistrationForm.

3.1 Срез favoritesSlice

Срез создан функцией createSlice, которая по описанию редюсеров автоматически генерирует объекты действий и строки их типов. Явно писать константы типов и функции-создатели действий не требуется. Состав среза приведён в таблице 3.

Таблица 3 – Состав среза favorites

Элемент	Назначение
initialState	Объект { ids: [] }: массив идентификаторов избранных спикеров
toggleFavorite(id)	Добавляет идентификатор в массив либо убирает его, если он уже там
clearFavorites()	Очищает список избранного целиком
selectFavoriteIds	Возвращает массив идентификаторов
selectFavoritesCount	Возвращает количество избранных спикеров
selectIsFavorite(id)	Возвращает признак принадлежности спикера к избранному

Редюсеры написаны так, будто состояние изменяется напрямую: в toggleFavorite вызываются методы push и splice. Это допустимо, потому что Redux Toolkit использует библиотеку Immer, которая перехватывает такие операции и собирает новый неизменяемый объект состояния. Исходное состояние при этом не меняется, требование неизменяемости соблюдено [1].

В хранилище лежат только идентификаторы, а данные спикеров остаются в модуле src/data/content.js. Такое разделение не даёт двум источникам истины разойтись: при изменении темы доклада править состояние хранилища не придётся.

Чтение состояния вынесено в селекторы. Компоненты не знают, что массив идентификаторов лежит в поле ids среза favorites, поэтому форму состояния можно изменить, не трогая разметку.

3.2 Корневое хранилище

Хранилище собрано функцией configureStore, которой передан объект редюсеров. Функция самостоятельно вызывает combineReducers, подключает Redux DevTools и набор стандартных промежуточных обработчиков, в том числе проверку на случайное изменение состояния и на несериализуемые значения в действиях.

Созданное хранилище передаётся в дерево компонентов компонентом Provider в файле main.jsx. Provider использует контекст React, поэтому любой компонент внутри дерева получает доступ к хранилищу хуками, а передавать его через props промежуточных уровней не нужно.

4 ПОДКЛЮЧЕНИЕ КОМПОНЕНТОВ

К хранилищу подключены два компонента. Их роли в обмене данными приведены в таблице 4.

Таблица 4 – Подключение компонентов к хранилищу

Компонент	Чтение через useSelector	Отправка через useDispatch
SpeakerCard	selectIsFavorite(id): признак отметки	toggleFavorite(id) по клику на звёздочку
Favorites	selectFavoritesCount, selectFavoriteIds	clearFavorites() по кнопке «Очистить»

Признак принадлежности спикера к избранному в локальном состоянии карточки не хранится: компонент читает его из хранилища при каждой перерисовке. Это выполняет требование условия о переносе логики из локального состояния на глобальный уровень.

Компонент Favorites не принимает ни одного свойства и не знает о существовании карточек. Карточка, в свою очередь, не знает о панели избранного. Связь между ними возникает только потому, что оба подписаны на один срез состояния, поэтому нажатие кнопки «Очистить» снимает отметки со всех карточек без прямого обращения к ним.

Отдельного внимания заслуживает выбор возвращаемого селектором значения. Хук useSelector после каждого отправленного действия вызывает селектор заново и сравнивает результат с предыдущим по ссылке. Если бы selectIsFavorite возвращал массив или объект, новая ссылка возникала бы при каждом вызове и карточка перерисовывалась бы на любое изменение хранилища. Селектор возвращает логическое значение, поэтому карточка перерисовывается только тогда, когда изменится отметка именно у неё.

Локальное состояние, не относящееся к избранному, намеренно оставлено в компонентах. Раскрытие блока FAQ и значения полей формы регистрации по-прежнему хранятся в хуке useState, поскольку эти данные нужны только внутри своего компонента и выносить их в глобальное хранилище нет причин.

5 РЕЗУЛЬТАТ РАБОТЫ ПРИЛОЖЕНИЯ

Исходное состояние интерфейса при пустом хранилище показано на рисунке 2.

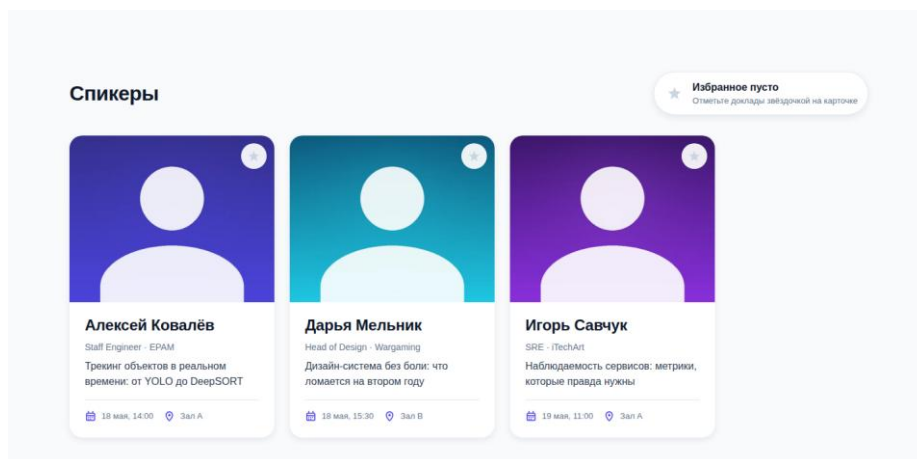


Рисунок 2 – Интерфейс при пустом списке избранного

Звёздочки на карточках не закрашены, панель над списком спикеров выводит сообщение о том, что избранное пусто, кнопка очистки скрыта. Все три значения вычислены из одного и того же среза состояния.

Состояние после добавления двух спикеров в избранное приведено на рисунке 3.

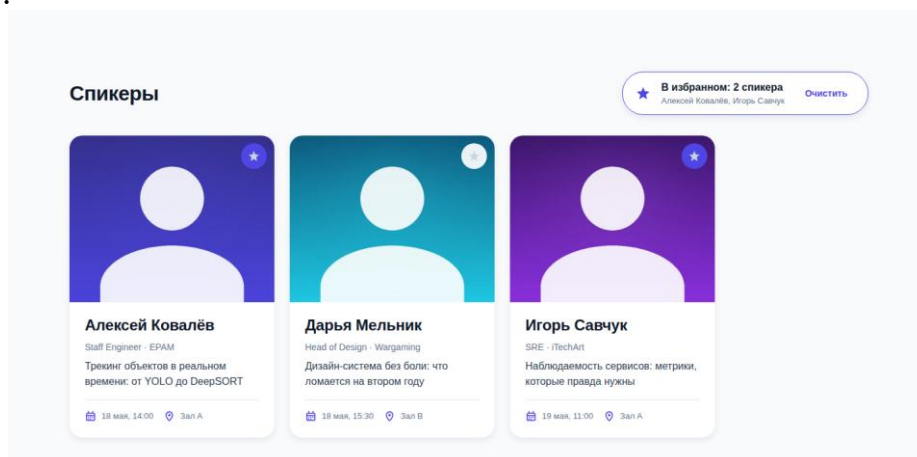


Рисунок 3 – Интерфейс после отправки двух действий toggleFavorite

Отмеченные карточки получили фирменный цвет кнопки, панель показывает количество избранных и перечисляет имена, появилась кнопка очистки. Панель обновилась не потому, что карточка сообщила ей об изменении, а потому, что хранилище вернуло новое состояние и все подписанные компоненты перерисовались.

Последовательность событий при нажатии на звёздочку выглядит так. Обработчик клика вызывает `dispatch` с результатом `toggleFavorite`, то есть с объектом действия, содержащим тип `favorites/toggleFavorite` и идентификатор спикера. Хранилище передаёт действие редюсеру среза, тот возвращает новое состояние с изменённым массивом идентификаторов. После этого хранилище уведомляет подписчиков, селекторы вычисляются заново, и React перерисовывает те компоненты, у которых результат селектора изменился.

Весь путь данных проходит через хранилище, поэтому порядок изменений воспроизводим и отслеживается в Redux DevTools.

6 ЗАПУСК ПРОЕКТА

Для запуска требуется Node.js версии 18 или новее. Последовательность команд:

- `npm install` - установка зависимостей, включая `@reduxjs/toolkit` и `react-redux`;
- `npm run dev` - запуск сервера разработки, по умолчанию на порту 5173;
- `npm run build` - сборка production-версии в каталог `dist`.

Если проект разворачивается поверх копии ЛР № 19, недостающие пакеты ставятся командой `npm install @reduxjs/toolkit react-redux`. Инструкции продублированы в файле `README.md` в корне репозитория, как требует раздел отчётности задания.

Для наблюдения за работой хранилища в браузер устанавливается расширение Redux DevTools. Дополнительной настройки проект не требует: `configureStore` подключает поддержку расширения в режиме разработки автоматически. После установки во вкладке расширения отображается список отправленных действий, состояние хранилища до и после каждого из них, а также возможность вернуться к любому предыдущему состоянию.

ЗАКЛЮЧЕНИЕ

В ходе работы в React-приложение из ЛР № 19 внедрено централизованное хранилище на Redux Toolkit. Создан срез `favorites` с двумя действиями и тремя селекторами, настроено корневое хранилище, корневой компонент обернут в `Provider`, к хранилищу подключены карточка спикера и панель избранного.

Практическая польза подхода видна на кнопке «Очистить»: панель избранного снимает отметки со всех карточек, не имея на них ни одной ссылки. При передаче состояния через `props` того же результата пришлось бы добиваться подъёмом состояния в общий родительский компонент и прокидыванием обработчиков через промежуточные уровни. Вынесение данных в хранилище оставило компоненты независимыми, а поток данных - прослеживаемым: каждое изменение проходит через действие и редюсер.